

Lab Assignment # 05

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #1 (Privacy in API Usage)

Task: Use an AI tool to generate a Python program that connects to a weather API.

Prompt: "Generate code to fetch weather data securely without exposing API keys in the code."

Expected Output:

- Original AI code (check if keys are hardcoded).
- Secure version using environment variables.

Output Screenshot:

```
weather.py X user_data.txt
weather.py > ...
1 #Task Description #1 (Privacy in API Usage
2 #generate a Python program that connects to aweather APIto fetch weather data securely without exposing API keys in the code
3 #Expected Output: • Original AI code (check if keys are hardcoded).Secure version using environment variables. using the API key = 6da90715355b
4
5 import os
6 import requests
7 from dotenv import load_dotenv
8
9 # Load variables from .env file
10 load_dotenv()
11
12 api_key = os.getenv('WEATHER_API_KEY')
13 city = input("Enter city name: ")
14 base_url = "http://api.openweathermap.org/data/2.5/weather?"
15 complete_url = f"{base_url}q={city}&appid={api_key}&units=metric"
16
17 response = requests.get(complete_url)
18 data = response.json()
19
20 if data.get("cod") == 200:
21     main = data["main"]
22     weather_desc = data["weather"][0]["description"]
23     print(f"City: {city}")
24     print(f"Temperature: {main['temp']}°C")
25     print(f"Weather: {weather_desc}")
26 else:
27     print("Error:", data.get("message", "City Not Found"))
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
• Enter city name: mumbai
City: mumbai
Temperature: 26.99°C
Weather: smoke
```

Task Description #2 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file. **Analyze:** Check if the AI stores sensitive data in plain text or without encryption.

Expected Output:

- Identified privacy risks.
- Revised version with encrypted password storage (e.g., hashing).

Output Screenshot:

```
31
32 #Task Description #2 (Privacy & Security in File Handling)
33 # Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file. Analyze: Check if the AI stores s
34 #Expected Output: • Identified privacy risks. • Revised version with encrypted password storage (e.g., hashing).
35
36 # Insecure version: stores password in plain text
37 import hashlib
38 from passlib.hash import bcrypt
39
40 def hash_password(password):
41     # Pre-hash with SHA-256 to avoid bcrypt length limit
42     sha256 = hashlib.sha256(password.encode()).hexdigest()
43     return bcrypt.hash(sha256)
44
45 def verify_password(stored_hash, entered_password):
46     sha256 = hashlib.sha256(entered_password.encode()).hexdigest()
47     return bcrypt.verify(sha256, stored_hash)
48
49 # Example usage
50 hashed = hash_password("mypassword123")
51 print("Stored hash:", hashed)
52 print("Verify:", verify_password(hashed, "mypassword123")) # ✓ True
53
54
55
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
Enter city name: mumbai
Temperature: 26.99°C
Weather: smoke
(.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "c:/Users/rgsga/OneDrive/Desktop/AI AC/w
eather.py"
Stored hash: $2b$12$CzSwiUxAfN5paxK4RIxHievda4KUTgdLw9euD6duHcotK.cZYjKTS
Verify: True
(.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC>
```

Task Description #3 (Transparency in Algorithm Design)

Objective: Use AI to generate an Armstrong number checking function with comments and explanations.

Instructions:

1. Ask AI to explain the code line-by-line.
2. Compare the explanation with code functionality.

Expected Output:

- Transparent, commented code.
- Correct, easy-to-understand explanation.

Output Screenshot:

```
57 #Task Description #3 (Transparency in Algorithm Design) Objective: Use AI to generate an Armstrong number checking function with comments and ex
58 #1. Ask AI to explain the code line-by-line. 2. Compare the explanation with code functionality.
59 #Expected Output: • Transparent, commented code. • Correct, easy-to-understand explanation.
60 def is_armstrong(number):
61
62
63
64     # Step 1: Convert the number to a string to easily count digits
65     num_str = str(number)
66
67     # Step 2: Find the number of digits
68     num_digits = len(num_str)
69
70     # Step 3: Initialize a variable to store the sum of powered digits
71     total = 0
72
73     # Step 4: Loop through each digit in the number
74     for digit in num_str:
75         # Convert digit back to integer and raise it to the power of num_digits
76         total += int(digit) ** num_digits
77
78     # Step 5: Compare the calculated sum with the original number
79     return total == number
80 print(is_armstrong(153)) # True (1^3 + 5^3 + 3^3 = 153)
81 print(is_armstrong(9474)) # True (9^4 + 4^4 + 7^4 + 4^4 = 9474)
82 print(is_armstrong(123)) # False
83
84 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
85
86 (.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "C:/Users/rgsga/OneDrive/Desktop/AI AC/w
eather.py"
87
88 • (.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "C:/Users/rgsga/OneDrive/Desktop/AI AC/w
eather.py"
89
90 True
91 True
92 False
93
94 C:\Users\rgsga\OneDrive\Desktop\AI AC> □
```

Task Description #4 (Transparency in Algorithm Comparison)

Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt: "Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ."

Expected Output:

- Code for both algorithms.
- Transparent, comparative explanation of their logic and efficiency.

Output Screenshot:

```
weather.py × user_data.txt
weather.py > ...
91 #Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).
92 #Prompt: "Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they diff
93 #Expected Output:
94 #• Code for both algorithms.
95 #• Transparent, comparative explanation of their logic and efficiency.
96
97
98 # Python Code: QuickSort vs BubbleSort
99 # QuickSort Implementation
100 def quicksort(arr):
101     """
102     QuickSort works using the 'divide and conquer' strategy:
103     1. Choose a pivot element (here, the middle element).
104     2. Partition the array into two halves:
105         - Left: elements smaller than pivot
106         - Right: elements greater than pivot
107     3. Recursively apply QuickSort to left and right halves.
108     4. Combine results into a sorted array.
109     """
110     if len(arr) <= 1: # Base case: arrays of length 0 or 1 are already sorted
111         return arr
112     else:
113         pivot = arr[len(arr) // 2] # Choose middle element as pivot
114         left = [x for x in arr if x < pivot] # Elements smaller than pivot
115         middle = [x for x in arr if x == pivot] # Elements equal to pivot
116         right = [x for x in arr if x > pivot] # Elements larger than pivot
117         return quicksort(left) + middle + quicksort(right)
118
119 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
120
121 (.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "C:/Users/rgsga/OneDrive/Desktop/AI AC/
eather.py"
122
123 • (.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "C:/Users/rgsga/OneDrive/Desktop/AI AC/
eather.py"
124
125 Original array: [34, 7, 23, 32, 5, 62]
126 QuickSort result: [5, 7, 23, 32, 34, 62]
127 BubbleSort result: [5, 7, 23, 32, 34, 62]
```

```
weather.py > ...
120 # BubbleSort Implementation
121 def bubblesort(arr):
122     """
123     BubbleSort works by repeatedly swapping adjacent elements:
124     1. Compare each pair of adjacent elements.
125     2. Swap them if they are in the wrong order.
126     3. Repeat passes until no swaps are needed (array is sorted).
127     """
128     n = len(arr)
129     for i in range(n):
130         # After each pass, the largest element "bubbles up" to the end
131         for j in range(0, n - i - 1):
132             if arr[j] > arr[j + 1]:
133                 # Swap if elements are in wrong order
134                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
135     return arr
136
137
138 # Example usage
139 data1 = [34, 7, 23, 32, 5, 62]
140 data2 = data1.copy()
141
142 print("Original array:", data1)
143 print("QuickSort result:", quicksort(data1))
144 print("BubbleSort result:", bubblesort(data2))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "c:/Users/rgsga/OneDrive/Desktop/eather.py"
(.venv) PS C:\Users\rgsga\OneDrive\Desktop\AI AC> & "C:/Users/rgsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "c:/Users/rgsga/OneDrive/Desktop/eather.py"
Original array: [34, 7, 23, 32, 5, 62]
QuickSort result: [5, 7, 23, 32, 34, 62]
BubbleSort result: [5, 7, 23, 32, 34, 62]
```

Key Differences

- QuickSort is much faster in practice because it reduces the problem size at each step by partitioning the array.
- BubbleSort is conceptually simple but inefficient, as it repeatedly scans the array and swaps elements, leading to quadratic time complexity.
- QuickSort is widely used in real-world applications, while BubbleSort is mostly used for educational purposes to demonstrate

◇ Efficiency Comparison

	$O(n \log n)$	$O(n)$
	$O(n \log n)$	$O(n^2)$
	$O(n^2)$	$O(n^2)$
	$O(\log n)$	$O(1)$

◇ Transparent Takeaways

- QuickSort is efficient and scalable:
It reduces the sorting problem size at each step, making it suitable for large datasets. Its recursive nature leverages divide-and-conquer, which is why it's widely used in practice.
- BubbleSort is simple but inefficient:
It's easy to understand and implement, but its repeated passes make it extremely slow for large inputs. It's mainly used for teaching sorting concepts, not real-world applications.
- Key Difference in Logic:
QuickSort *divides the array into smaller problems*, while BubbleSort *repeatedly pushes larger elements to the end*.
This difference in strategy explains why QuickSort achieves logarithmic efficiency while BubbleSort suffers quadratic inefficiency

Task Description #5 (Transparency in AI Recommendations)

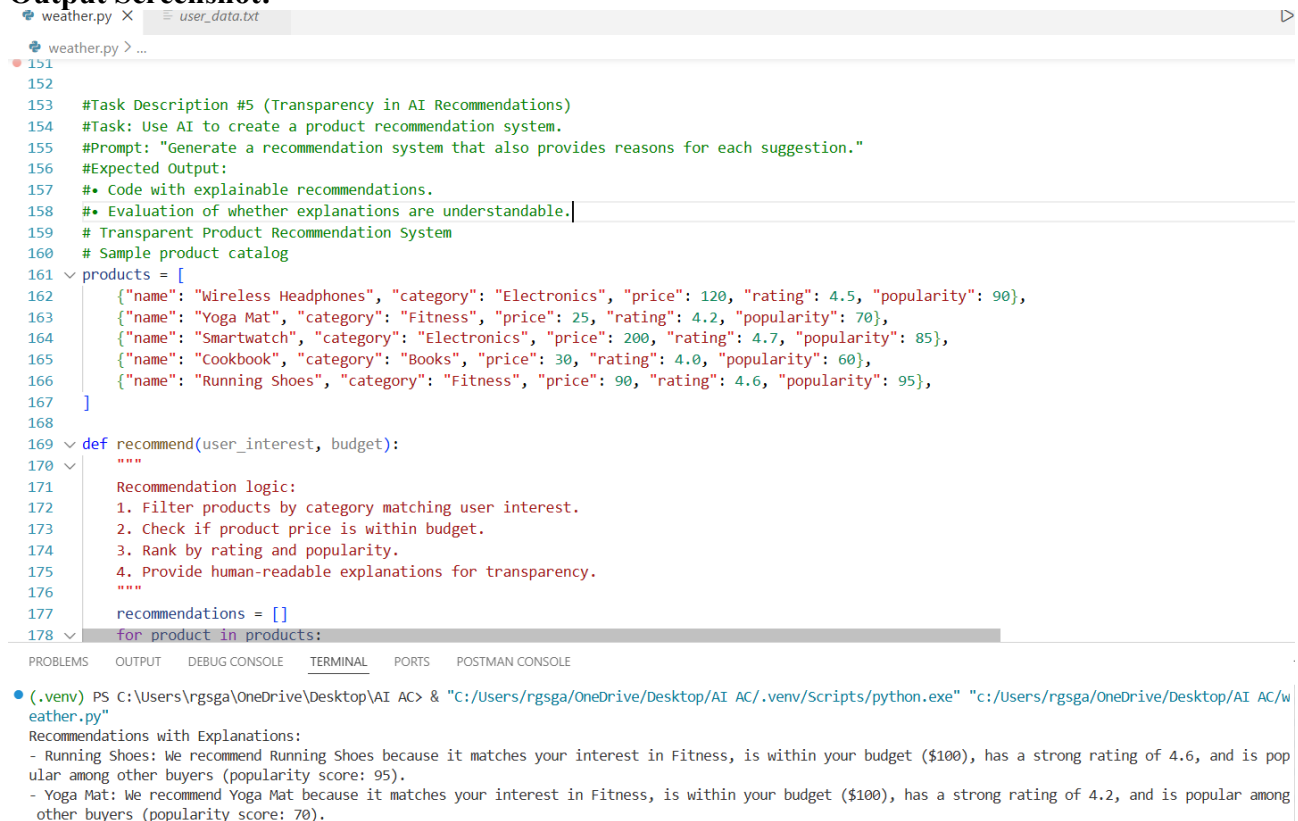
Task: Use AI to create a product recommendation system.

Prompt: "Generate a recommendation system that also provides reasons for each suggestion."

Expected Output:

- Code with explainable recommendations.
- Evaluation of whether explanations are understandable.

Output Screenshot:



```
weather.py × user_data.txt
weather.py > ...
151
152
153 #Task Description #5 (Transparency in AI Recommendations)
154 #Task: Use AI to create a product recommendation system.
155 #Prompt: "Generate a recommendation system that also provides reasons for each suggestion."
156 #Expected Output:
157 #• Code with explainable recommendations.
158 #• Evaluation of whether explanations are understandable.
159 # Transparent Product Recommendation System
160 # Sample product catalog
161 products = [
162     {"name": "Wireless Headphones", "category": "Electronics", "price": 120, "rating": 4.5, "popularity": 90},
163     {"name": "Yoga Mat", "category": "Fitness", "price": 25, "rating": 4.2, "popularity": 70},
164     {"name": "Smartwatch", "category": "Electronics", "price": 200, "rating": 4.7, "popularity": 85},
165     {"name": "Cookbook", "category": "Books", "price": 30, "rating": 4.0, "popularity": 60},
166     {"name": "Running Shoes", "category": "Fitness", "price": 90, "rating": 4.6, "popularity": 95},
167 ]
168
169 def recommend(user_interest, budget):
170     """
171     Recommendation logic:
172     1. Filter products by category matching user interest.
173     2. Check if product price is within budget.
174     3. Rank by rating and popularity.
175     4. Provide human-readable explanations for transparency.
176     """
177     recommendations = []
178     for product in products:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
(.venv) PS C:\Users\rngsga\OneDrive\Desktop\AI AC> & "C:/Users/rngsga/OneDrive/Desktop/AI AC/.venv/Scripts/python.exe" "c:/Users/rngsga/OneDrive/Desktop/AI AC/weather.py"
Recommendations with Explanations:
- Running Shoes: We recommend Running Shoes because it matches your interest in Fitness, is within your budget ($100), has a strong rating of 4.6, and is popular among other buyers (popularity score: 95).
- Yoga Mat: We recommend Yoga Mat because it matches your interest in Fitness, is within your budget ($100), has a strong rating of 4.2, and is popular among other buyers (popularity score: 70).
```

Evaluation of Explanations

- **Understandability:**

The explanations are clear and human-friendly. They explicitly state:

- The product name.
- Why it matches the user's interest (category alignment).
- Why it fits the budget.
- Why it's considered good (rating + popularity).

- **Transparency:**

Users can see the criteria used (interest, budget, rating, popularity). This avoids the "black box" problem of AI recommendations.

- **Strengths:**

- Easy to read and interpret.
- Shows multiple factors (budget, rating, popularity).
- Provides confidence in why the product was chosen.

- **Possible Improvements:**

- Could add trade-offs (e.g., "slightly higher price but better rating").
- Could personalize further (e.g., "similar to items you liked before").